

Sir Syed CASE Institute of Technology

Visual Programming

Lecture 2: Introduction to C#

3.00 (2 + 1) Credit Hours, Spring 2025

Instructor: Mr. Hamid Ashfaq

Email: hamid.Ashfaq@case.edu.pk

In previous lecture

Introduction to Visual Studio

.Net framework

Managed and Unmanaged code

In today's lecture

Introduction to Visual Studio

Installation

Basics of C#

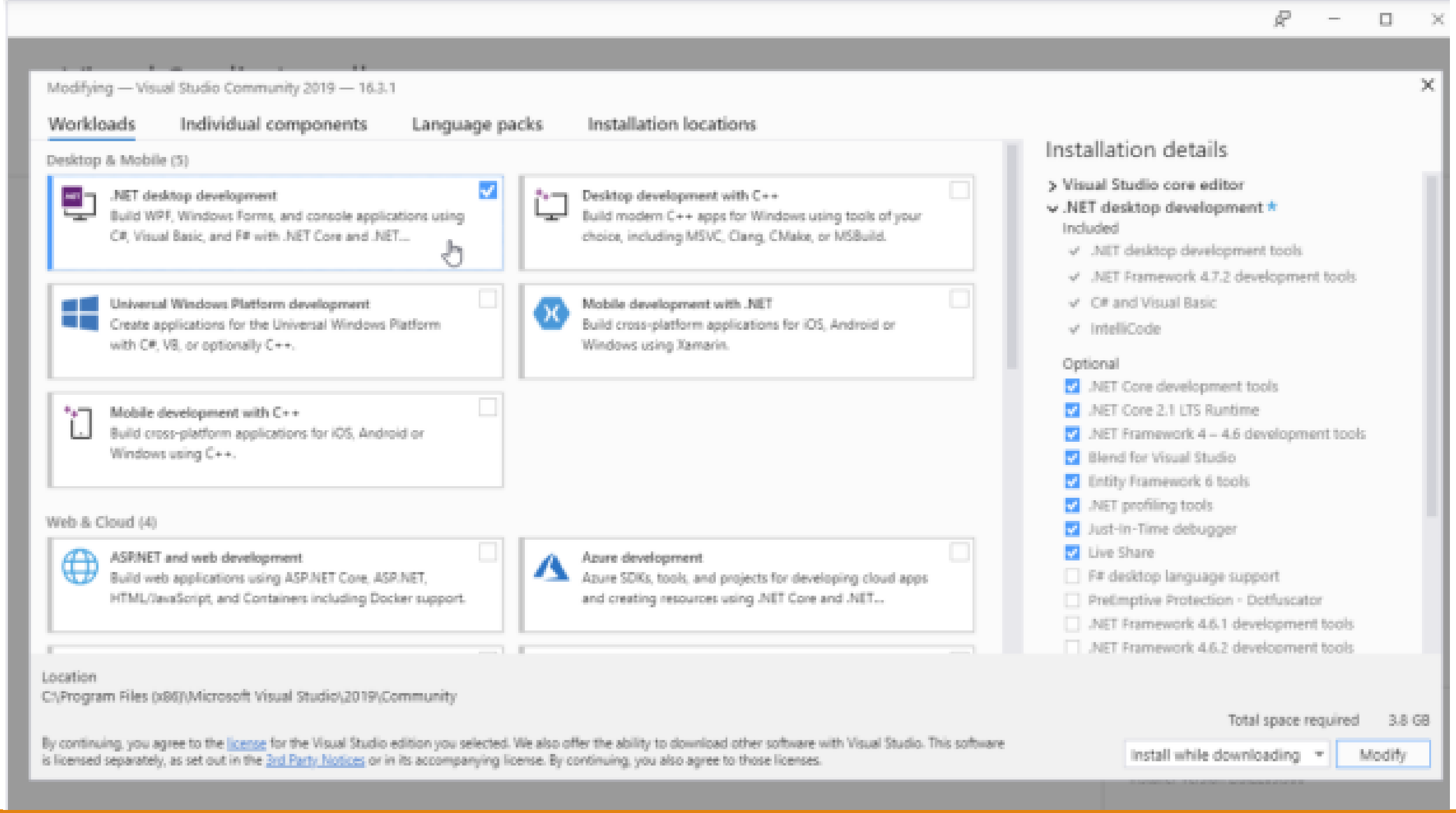
C# string

C# string methods

Visual Studio Installation

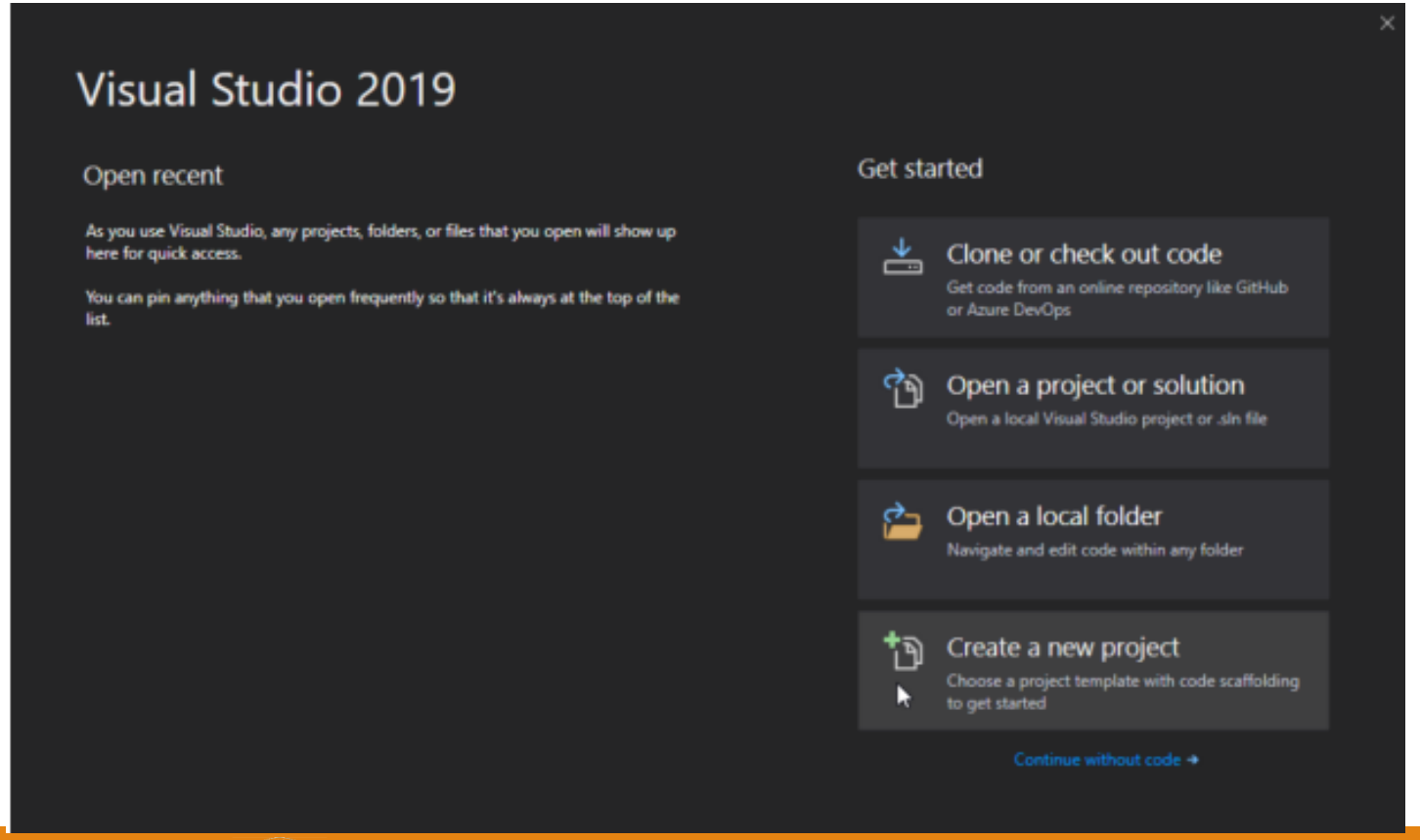
Once the Visual Studio Installer is downloaded and installed, choose the .NET workload and click on the Modify/Install button.

After the installation is complete, click on the Launch button to get started with Visual Studio.


Modifying — Visual Studio Community 2019 — 16.3.1
Workloads
Individual components
Language packs
Installation locations
Desktop & Mobile (3)
.NET desktop development
Build WPF, Windows Forms, and console applications using C#, Visual Basic, and F# with .NET Core and .NET...
Desktop development with C++
Build modern C++ apps for Windows using tools of your choice, including MSVC, Clang, CMake, or MSBuild.
Universal Windows Platform development
Create applications for the Universal Windows Platform with C#, VB, or optionally C++.
Mobile development with .NET
Build cross-platform applications for iOS, Android or Windows using Xamarin.
Mobile development with C++
Build cross-platform applications for iOS, Android or Windows using C++.
Web & Cloud (4)
ASP.NET and web development
Build web applications using ASP.NET Core, ASP.NET, HTML/JavaScript, and Containers including Docker support.
Azure development
Azure SDKs, tools, and projects for developing cloud apps and creating resources using .NET Core and .NET...
Installation details
Visual Studio core editor
.NET desktop development *
Included
.NET desktop development tools
.NET Framework 4.7.2 development tools
C# and Visual Basic
IntelliCode
Optional
.NET Core development tools
.NET Core 2.1 LTS Runtime
.NET Framework 4 – 4.6 development tools
Blend for Visual Studio
Entity Framework 6 tools
.NET profiling tools
Just-In-Time debugger
Live Share
F# desktop language support
PreEmptive Protection - Dotfuscator
.NET Framework 4.6.1 development tools
.NET Framework 4.6.2 development tools
Location
C:\Program Files (x86)\Microsoft Visual Studio\2019\Community
Total space required 3.8 GB
By continuing, you agree to the license for the Visual Studio edition you selected. We also offer the ability to download other software with Visual Studio. This software is licensed separately, as set out in the 3rd Party Notices or in its accompanying license. By continuing, you also agree to those licenses.
Install while downloading
Modify

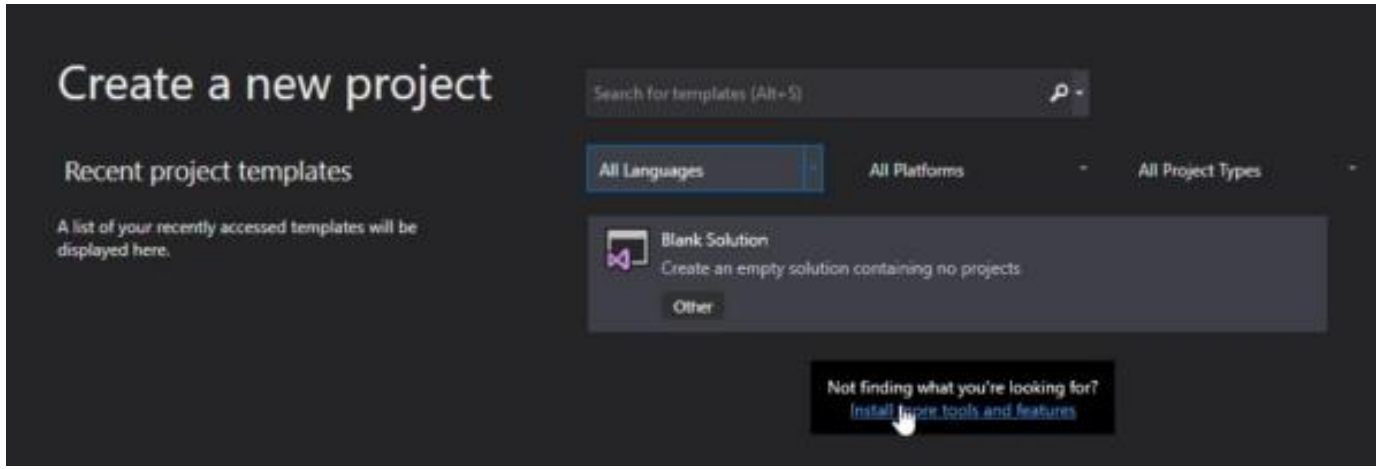
Introduction to Visual Studio

On the start window, choose Create a new project

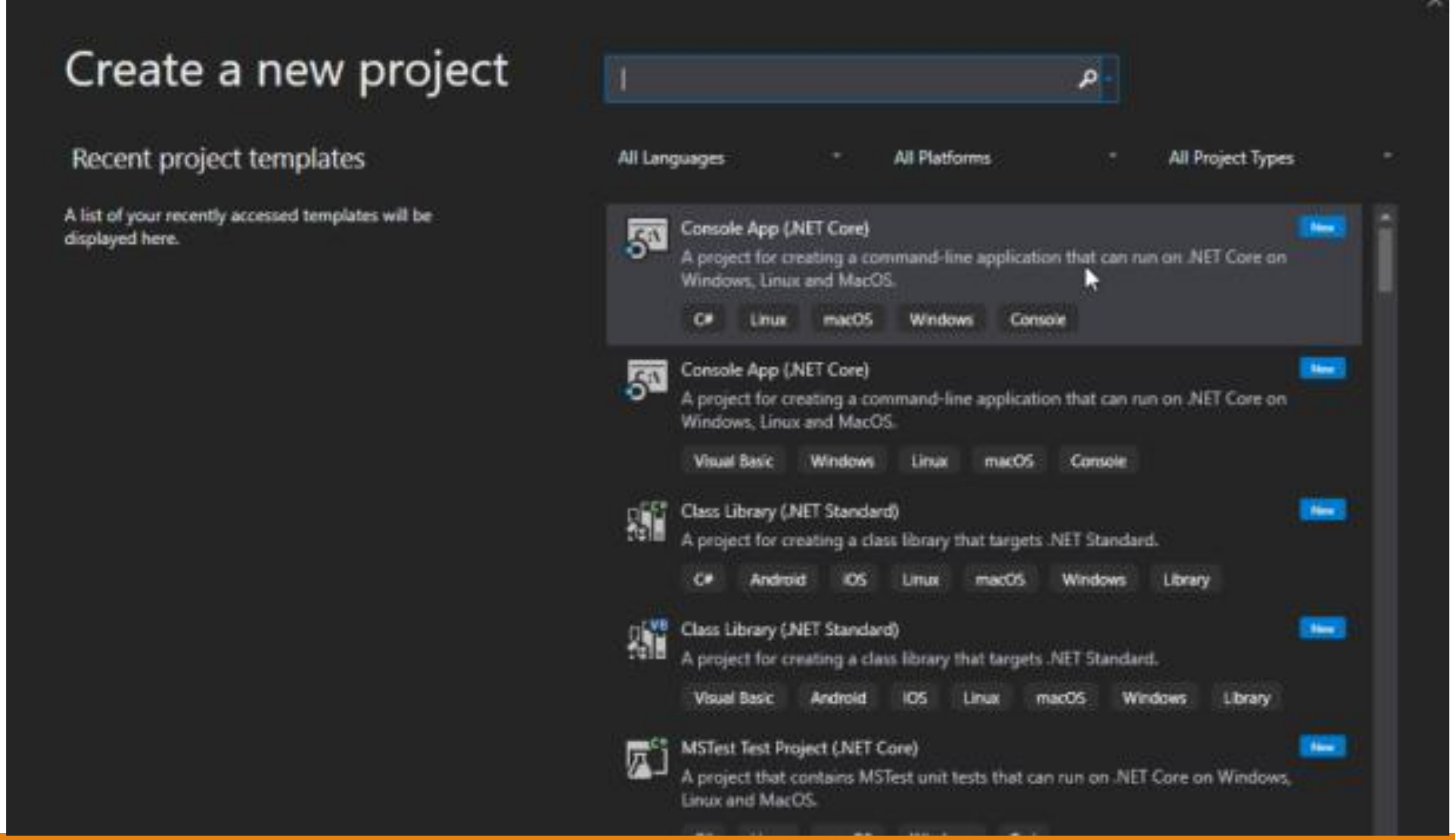


Introduction to Visual Studio

Then click on the "Install more tools and features" button



Choose "Console App (.NET Core)" from the list and click on the Next button


Create a new project
Recent project templates
A list of your recently accessed templates will be displayed here.
All Languages
All Platforms
All Project Types
Console App (.NET Core)
A project for creating a command-line application that can run on .NET Core on Windows, Linux and MacOS.
C# Linux macOS Windows Console
Console App (.NET Core)
A project for creating a command-line application that can run on .NET Core on Windows, Linux and MacOS.
Visual Basic Windows Linux macOS Console
Class Library (.NET Standard)
A project for creating a class library that targets .NET Standard.
C# Android iOS Linux macOS Windows Library
Class Library (.NET Standard)
A project for creating a class library that targets .NET Standard.
Visual Basic Android iOS Linux macOS Windows Library
MSTest Test Project (.NET Core)
A project that contains MSTest unit tests that can run on .NET Core on Windows, Linux and MacOS.

C# Code

using System means that we can use classes from the System namespace.

namespace is a used to organize your code, and it is a container for classes and other namespaces

class is a container for data and methods, which brings functionality to your program. Every line of code that runs in C# must be inside a class.

Console is a class of the System namespace

C# Code

```
using System;
namespace Csharp
{
 class Program
 {
 static void Main(string[] args)
 {
 Console.WriteLine(“C# Lecture!");
 }
 }
}
```

Output statement

The most common method to output something in C# is WriteLine(), but you can also use Write()

The difference is that WriteLine() prints the output on a new line each time, while Write() prints on the same line (note that you should remember to add spaces when needed, for better readability.

C# Comments

Comments can be used to explain C# code, and to make it more readable.

It can also be used to prevent execution when testing alternative code.

Single-line comments start with two forward slashes (//).

Any text between // and the end of the line is ignored by C# (will not be executed).

Multi-line comments start with /* and ends with */.

Any text between /* and */ will be ignored by C#.

C# variables

Variables are containers for storing data values.

In C#, there are different types of variables (defined with different keywords), for example:

- **int** - stores integers (whole numbers), without decimals, such as 123 or -123
- **double** - stores floating point numbers, with decimals, such as 19.99 or -19.99
- **char** - stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes
- **string** - stores text, such as "Hello World". String values are surrounded by double quotes
- **bool** - stores values with two states: true or false
- **long** – stores large value for an integer

C# constants

However, you can add the const keyword if you don't want others (or yourself) to overwrite existing values (this will declare the variable as "constant", which means unchangeable and read-only)

The const keyword is useful when you want a variable to always store the same value, so that others (or yourself) won't mess up your code. An example that is often referred to as a constant, is PI

(3.14159...).

You cannot declare a constant variable without assigning the value. If you do, an error will occur: A const field requires a value to be provided.

const int myNum = 50;

myNum = 20; // error

C# typecasting

Type casting is when you assign a value of one data type to another type.

In C#, there are two types of casting:

• **Implicit Casting** (automatically) - converting a smaller type to a larger type size

char -> int -> long -> float -> double

• **Explicit Casting** (manually) - converting a larger type to a smaller size type

double -> float -> long -> int -> char

Implicit typecasting

Implicit casting is done automatically when passing a smaller size type to a larger size type.

```
int myInt = 9;

double myDouble = myInt; // Automatic casting: int to double

Console.WriteLine(myInt); // Outputs 9

Console.WriteLine(myDouble); // Outputs 9
```

Explicit typecasting

Explicit casting must be done manually by placing the type in parentheses in front of the value

```
double myDouble = 9.78;

int myInt = (int) myDouble; // Manual casting: double to int

Console.WriteLine(myDouble); // Outputs 9.78

Console.WriteLine(myInt); // Outputs 9
```

Type Conversion Methods

It is also possible to convert data types explicitly by using built-in methods, such as

Convert.ToBoolean

Convert.ToDouble

Convert.ToString

Convert.ToInt32 (int)

and Convert.ToInt64 (long)

Type Conversion Methods

```
int myInt = 10;
double myDouble = 5.25;
bool myBool = true;
Console.WriteLine(Convert.ToString(myInt)); // convert int to string
Console.WriteLine(Convert.ToDouble(myInt)); // convert int to double
Console.WriteLine(Convert.ToInt32(myDouble)); // convert double to int
Console.WriteLine(Convert.ToString(myBool)); // convert bool to string
```

C# user input

use Console.ReadLine() to get user input.

Example the user can input his or hers username, which is stored in the variable userName. Then we print the value of username

```
Console.WriteLine("Enter username:");
string userName = Console.ReadLine();
Console.WriteLine("Username is: " + userName);
```

C# user input

The Console.ReadLine() method returns a string. Therefore, you cannot get information from another data type, such as int. The following program will cause an error

```
Console.WriteLine("Enter your age:");

int age = Console.ReadLine();

Console.WriteLine("Your age is: " + age);
```

C# user input

Solution:

```
Console.WriteLine("Enter your age:");
int age = Convert.ToInt32(Console.ReadLine());
Console.WriteLine("Your age is: " + age);
```

C# Operators

Arithmetic Operators: are used to perform common mathematical operations (+, - , *, /, %)

Increment/Decrement Operators: are used to increase/decrease variable value by 1. (++, --)

C# Assignment Operators: is used to assign value (=, +=,-=,*=,/=,%=)

C# Comparison Operators: are used to compare values (==,!=, >,<,>=,<=)

C# Logical Operators: &&, ||, !

C# Math

Math.Max(x,y)

The Math.Max(x,y) method can be used to find the highest value of x and y:

Math.Max(5, 10);

Math.Min(x,y)

The Math.Min(x,y) method can be used to find the lowest value of of x and y:

Math.Min(5, 10);

Math.Sqrt(x)

The Math.Sqrt(x) method returns the square root of x:

Math.Sqrt(64);

C# Math

Math.Abs(x)

The Math.Abs(x) method returns the absolute (positive) value of x:

```
Math.Abs(-4.7);
```

String

Strings are used for storing text.

A string variable contains a collection of characters surrounded by double quotes.

```
string greeting = "Hello";
Console.WriteLine(greeting);
```



String properties

Length: A string in C# is actually an object, which contain properties and methods that can perform certain operations on strings. For example, the length of a string can be found with the Length property.

```
string txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
Console.WriteLine("The length of the txt string is: " + txt.Length);
```

String Methods

ToUpper(): which returns a copy of the string converted to uppercase

ToLower(): which returns a copy of the string converted to lowercase

```
string txt = “Visual Programming”;
Console.WriteLine(txt.ToUpper());   // Outputs "VISUAL PROGRAMMING"
Console.WriteLine(txt.ToLower());   // Outputs "visual programming"
```

String Methods

Concat(): method to concatenate/combine two strings. Its working is similar to + operator

```
string txt1 = “Visual ”;
string txt2 = “Programming ”;
string course =txt1 + text2;
Console.WriteLine(course);   // Outputs "Visual Programming“
string subject = string.Concat(txt1,txt2);
Console.WriteLine(subject);   // Outputs "Visual Programming"
```

String Methods

Another option of string concatenation, is string interpolation, which substitutes values of variables into placeholders in a string. Note that you do not have to worry about spaces, like with concatenation.

```
string txt1 = “Visual ”;

string txt2 = “Programming ”;

Console.WriteLine($”Course Name is: {txt1} {txt2}”);


// Outputs " Course Name is: Visual Programming“
```

String Methods

IndexOf(): returns the index number of value where it is placed.

```
string txt1 = “Visual ”;
Console.WriteLine(IndexOf(“u”));          // Outputs  “3”

Console.WriteLine(txt1[2]);               // Outputs  “s”
```

String Methods

Substring(): which extracts the characters from a string, starting from the specified character position/index, and returns a new string. This method is often used together with IndexOf() to get the specific character position

```
string txt1 = “Visual ”;
int index= txt1.IndexOf(“u”);                    // Store  “3”
string txt2 = txt2.Substring(index);
Console.WriteLine(txt2);                         // Outputs  “ual”
```

In Next lecture

Conditional statements

Loops